

Project Checkpoint Report:
Mobile Application - TownHappening

Danielle Kemp

Department of Informatics, Indiana University

INFO-C 342 – Mobile Application Development

June 23, 2026

Project Checkpoint Report

GUI Design

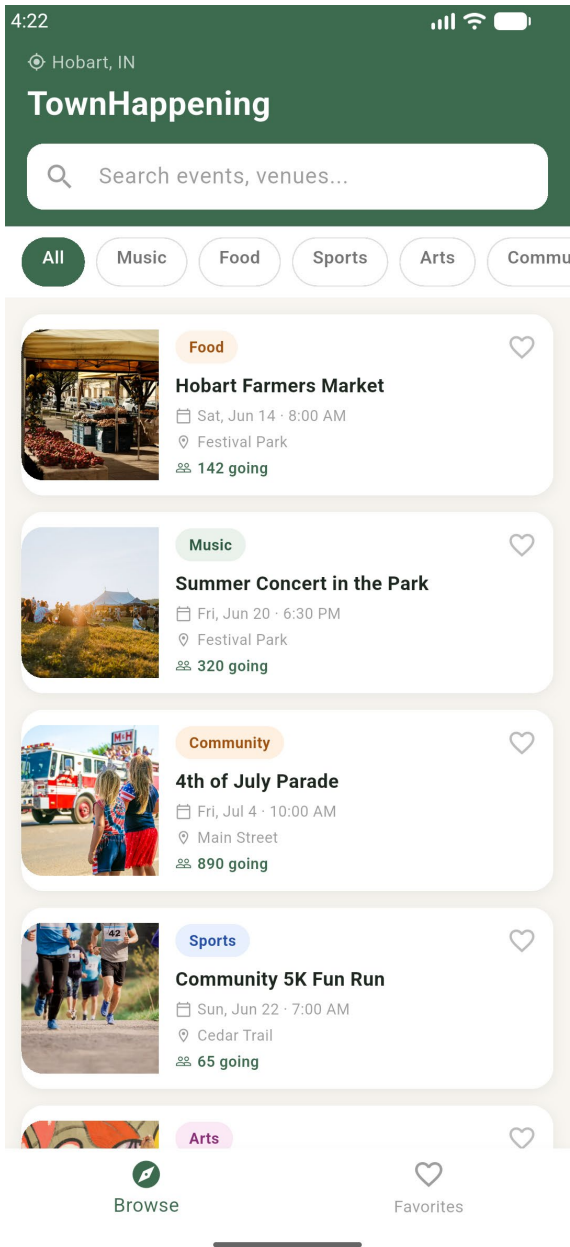


Figure 1- Home Screen

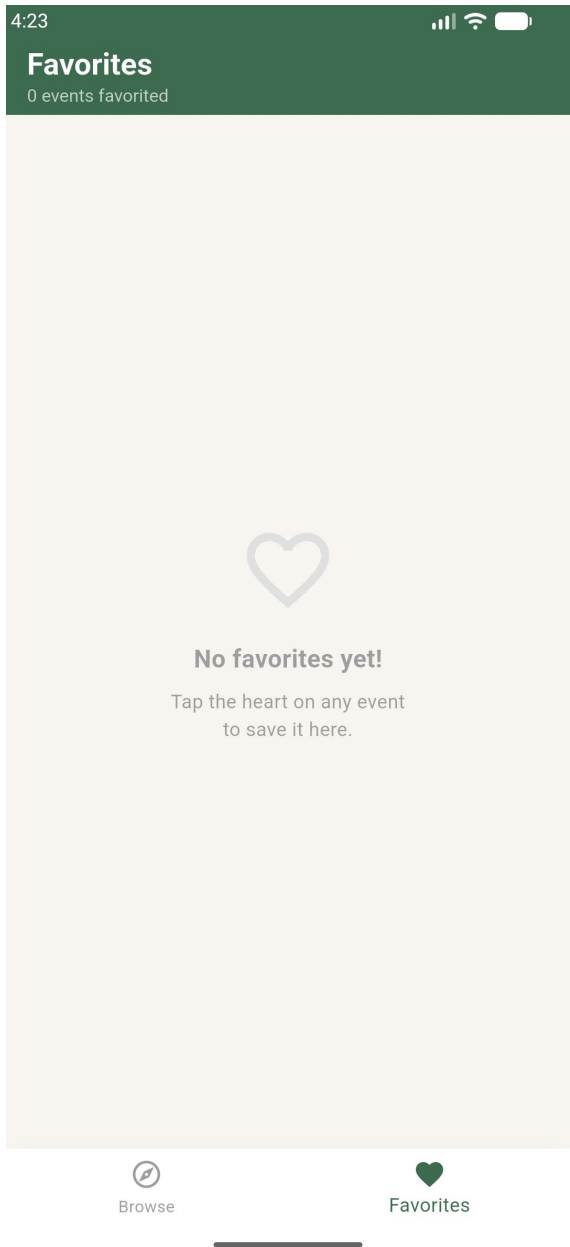


Figure 2 - Favorites Screen

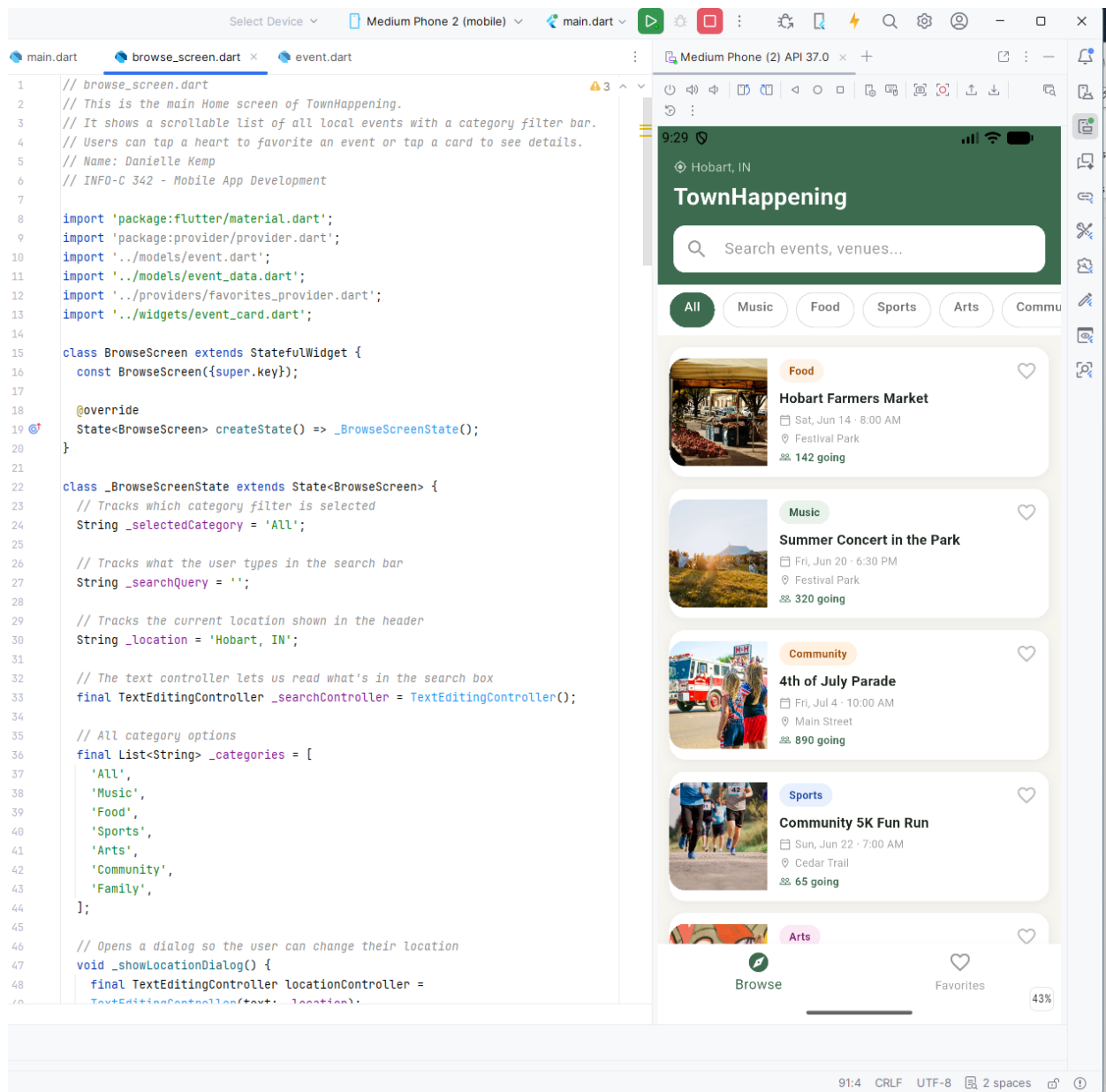


Figure 3- Home Screen with code snippets

The user interface for TownHappening has been implemented using Flutter widgets, and the core application screens are nearly fully functional. The design is visually consistent across screens and follows a green theme, which was chosen to represent community engagement and a welcoming local environment.

The Home Screen displays a scrollable list of event cards using `ListView.builder`. It also includes a search bar that filters events by title and location. Category filter chips were also added to allow users to quickly narrow down events by type, improving usability and navigation.

Currently, the Favorites Screen displays all of the saved events and it includes an empty state when no favorites exist. This screen updates automatically when users tap the heart icon on any event card, reflecting changes across the application in real time.

Research, Learning, and Experimentation

During the development of TownHappening, several Flutter and Dart concepts were researched, tested, and implemented. The Flutter layout system (Row, Column, Expanded, and Container) was used to build the event cards and the detail screens. This was one of the most significant learning areas, and it improved my understanding of responsive UI design in Flutter.

Navigation was implemented using `Navigator.push` and `MaterialPageRoute`. The transition between the Browse Screen and Event Detail Screen was designed to provide a smooth user experience, allowing users to access full event details easily.

State management across multiple screens was implemented using `Provider` (`ChangeNotifier`). This was used to ensure that the favorite status of events remains consistent across the Browse, Favorites, and Detail screens.

Since no backend database was used, event favorites were stored locally using `SharedPreferences`. This allowed event IDs to persist even after the application was closed and reopened. Efficient list rendering was achieved using `ListView.builder`, improving performance when displaying multiple events.

External packages used in this project include:

- provider
- shared_preferences
- share_plus

Learning Sources

- Flutter official documentation
 - <https://docs.flutter.dev/>
 - <https://docs.flutter.dev/data-and-backend/state-mgmt/simple>
- The pub.dev package documentation for Provider and SharedPreferences
 - <https://pub.dev/packages/provider>
 - **Shared Preferences:**
<https://www.youtube.com/watch?v=szOllHT1S7Y>
- In-class labs and assignments
- YouTube Tutorials
 - **Flutter Share Files & Text From App:**
<https://www.youtube.com/watch?v=7gjq9MFD0I>
 - **Listview & Builder :** <https://www.youtube.com/watch?v=k1LxTsmAURU>
 - **Navigator push:**
<https://www.youtube.com/watch?v=3F3E-gZ5BF0>
 - **Flutter Vertical ListView:**
<https://www.youtube.com/watch?v=uwF0veucf5Q>

Changes From Proposal

Most of the original ideas from the proposal were maintained. Figma wireframes helped guide the initial UI structure and reduced design iteration time during development.

One key change was the decision to use Provider for state management. This approach was selected because it effectively handled shared state across multiple screens, including Browse, Favorites, and Event Detail screens.

Additionally, while Firebase was introduced in class, the current implementation uses a local hardcoded dataset for simplicity. This allowed more focus on UI development and state management concepts during early development. A future improvement would be integrating Firebase Firestore for dynamic event storage.

Navigation was also simplified by using MaterialPageRoute instead of named routes to reduce complexity during development.

Progress Status

At this point, about 40-50% or more of the application is completed.

Completed Features

- Browse screen with full movement of tapping and scrolling
- Event model and a structured dataset
- Favorites system is completed on two screens out of three screens
- Saving data on the device even after the app closes (SharedPreferences)
(including favorites and IDs)
- Bottom navigation

- Category filtering
- Search bar (fully functional)
- Added images successfully to two screens

In Progress

- Event Card Detail widget / screen
- Improving consistency of event card design and spacing & adding photos
- Location change feature on home screen
- Continue to work on smooth transitions
- Fixing minor bug issues and debugging issues during testing

Difficulties Encountered

Before development began, an issue occurred with the emulator running out of storage space, which required switching to a different virtual device. This caused an initial delay in development.

During development, Flutter build caching issues occurred, requiring project rebuilds and clearing temporary build files. These issues were resolved through troubleshooting using documentation and AI-assisted debugging support.

Additional minor issues included syntax errors, missing imports, and challenges passing event data between screens. These were resolved through iterative debugging and reviewing Flutter documentation.

Another challenge involved UI layout consistency, particularly with container sizing in event cards and filter components.

Ensuring data persistence was also a key challenge. It was important that user favorites remained saved after closing the application. This was successfully resolved using `SharedPreferences` to store event IDs locally. Future improvement will focus on integrating `Firebase Firestore` for more scalable data management.